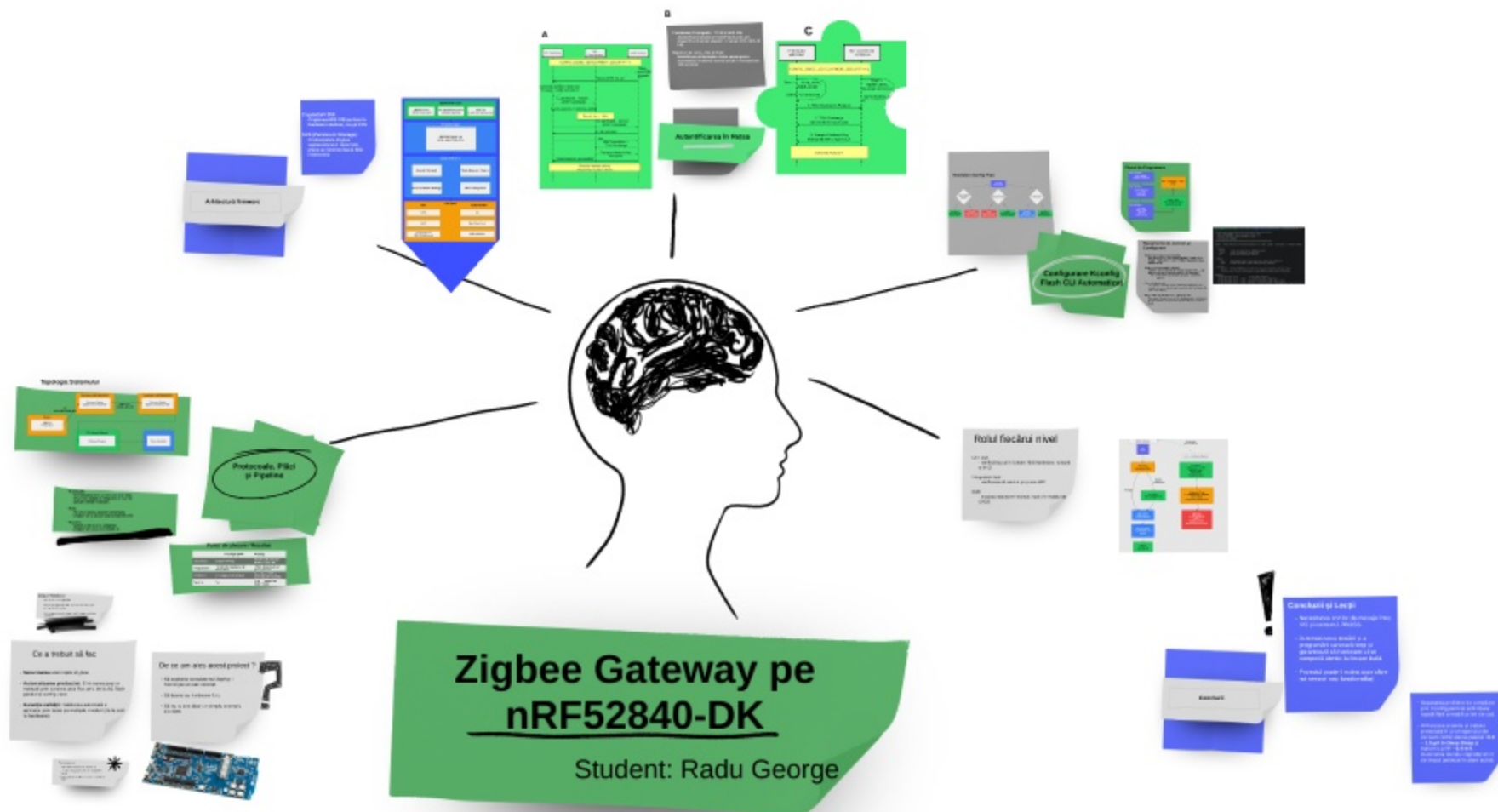
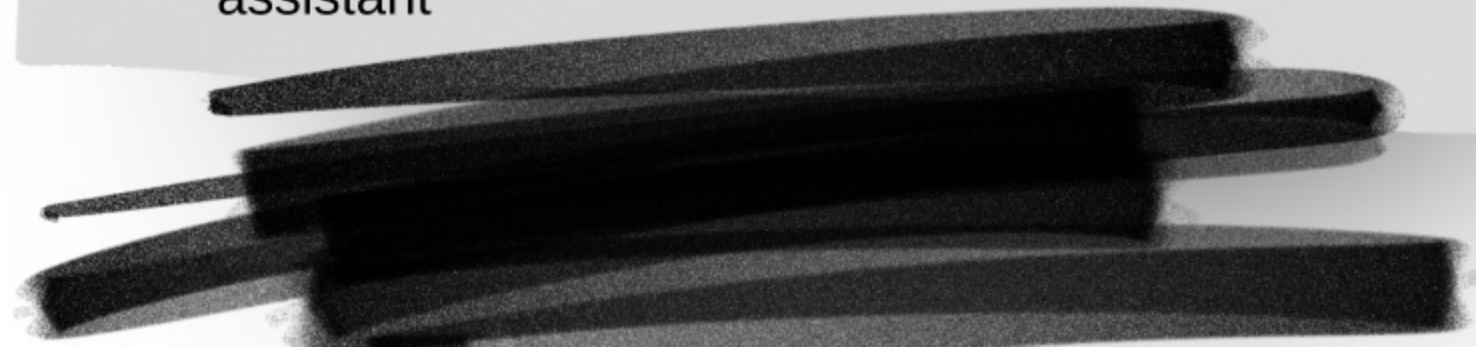




Zigbee Gateway pe nRF52840-DK

Student: Radu George

Scopul Proiectului:

- Un senzor de temperatură
 - Două sau mai multe plăci care comunică wireless prin protocolul zigbee
 - Un computer care primește datele integrat cu home assistant
- 

Ce a trebuit să fac

Implementarea unei rețele Zigbee

Automatizarea producției: Eliminarea pașilor manuali prin crearea unui flux unic de build, flash și configurare.

Verificarea calității: Validarea automată a funcționării prin teste pe multiple niveluri (de la cod la hardware)

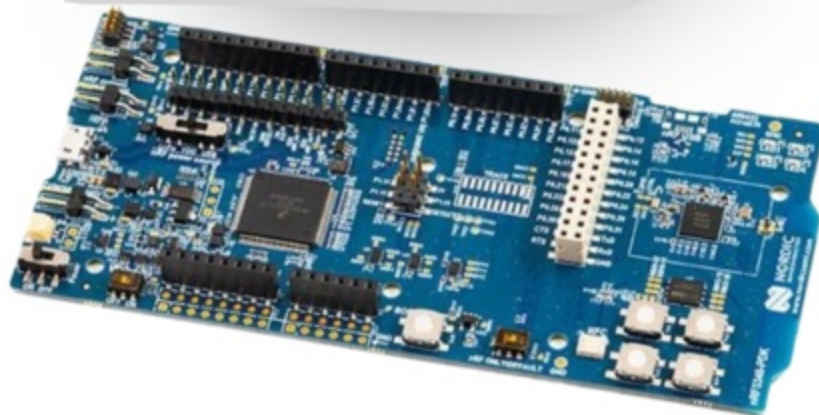
Ce am rezolvat

- Rețea Zigbee, funcțională pe hardware fizic
- Un singur script ajutător: compilare + programare + validare
- Testare pe etape, până la comunicație completă între 2 plăci



De ce am ales acest proiect ?

- Să explorez ecosistemul Zephyr / Nordic pe un caz concret
- Să lucrez cu hardware fizic
- Să nu ruleze doar un simplu exemplu din SDK



Ce a trebuit să fac

- **Securizarea** unei rețele Zigbee
- **Automatizarea producției:** Eliminarea pașilor manuali prin crearea unui flux unic de build, flash paralel și configurare.
- **Garanția calității:** Validarea automată a aplicației prin teste pe multiple niveluri (de la cod la hardware)

De ce

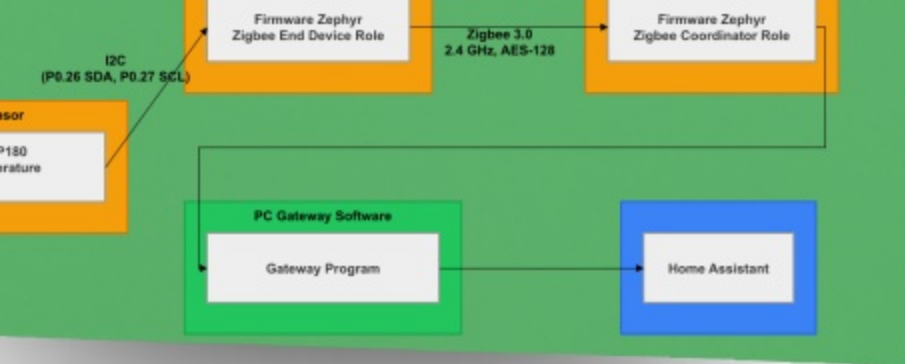
- Să exp Nordic
- Să luc
- Să nu din SD



Ce am rezolvat

- Rețea Zigbee, funcțională pe hardware fizic
- Un singur script ajutător: compilare + programare + validare
- Testare pe etape, până la comunicație completă între 2 plăci





Protocoale, Plăci și Pipeline

- **Protocoale:**
Sincronizarea între achiziția de date (I2C), stiva radio Zigbee și integrarea la nivel de aplicație Home Assistant.
- **Plăci:**
Au roluri opuse, asociere securizată, programare și testare coordonată între ele
- **Pipeline:**
Construit de la zero: compilare, programare, asociere, testare, CI

Punct de plecare / Rezultat

	Exemple SDK	Proiect
Securitate	Global Link Key	Install Code unic per device + AES-128
Programare	~4 comenzi manuale, căi hardcodeate	.\flash_devices.sh all --prod --persist
Firmware	Un singur profil de build	Dev / Prod / E2E — controlat prin Kconfig

- **Protocoloale:**

Sincronizarea între achiziția de date (I2C), stiva radio Zigbee și integrarea la nivel de aplicație Home Assistant.

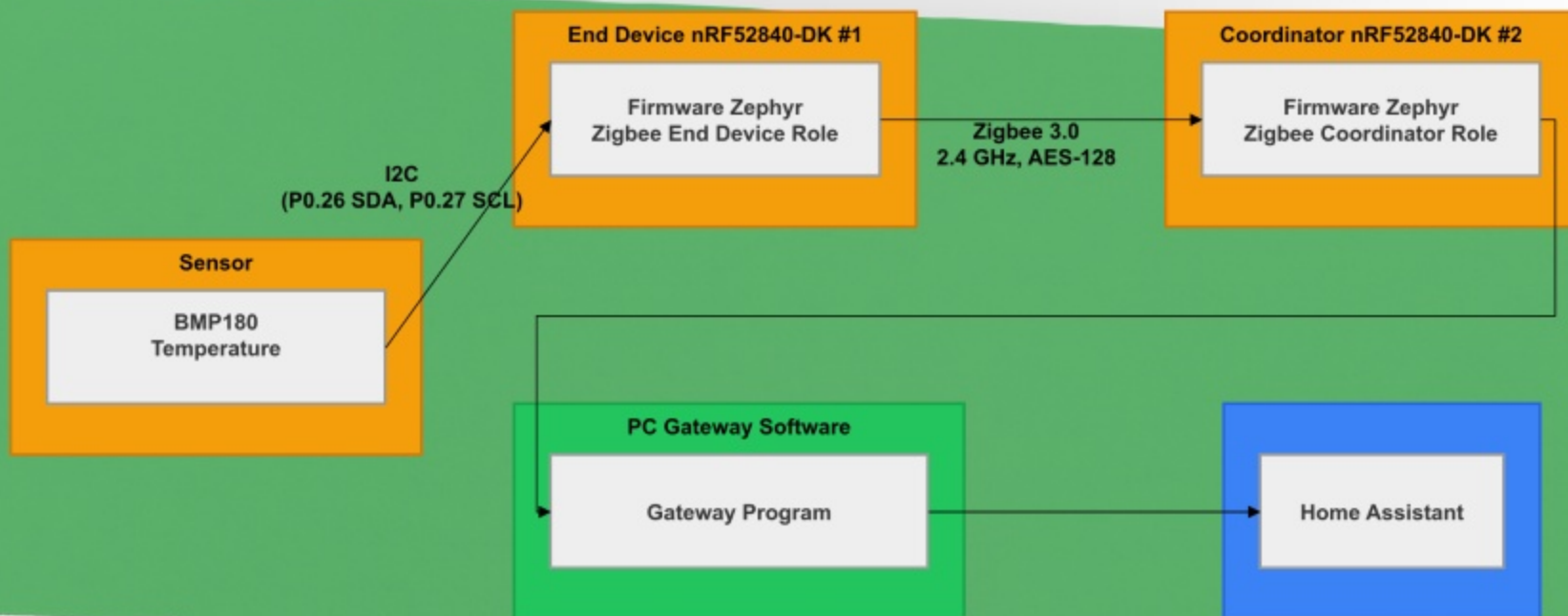
- **Plăci:**

Au roluri opuse, asociere securizată, programare și testare coordonată între ele

- **Pipeline:**

Construit de la zero: compilare, programare, asociere, testare, CI

Topologia Sistemului



Punct de plecare / Rezultat

	Exemple SDK	Proiect
Securitate	Global Link Key	Install Code unic per device + AES-128
Programare	~4 comenzi manuale, căi hardcodate	<code>./flash_devices.sh all --prod --persist</code>
Firmware	Un singur profil de build	Dev / Prod / E2E — controlat prin Kconfig
Testare	Zero	Unit → Integration → E2E + CI/CD

Arhitectură firmware

CryptoCell-310:

Criptarea AES-128 se face în hardware dedicat, nu pe CPU

NVS (Persistent Storage):

Credențialele Zigbee supraviețuiesc repornirii, placa se reconectează fără intervenție



CryptoCell-310:

Criptarea AES-128 se face în hardware dedicat, nu pe CPU

NVS (Persistent Storage):

Credențialele Zigbee supraviețuiesc repornirii, placa se reconectează fără intervenție



A

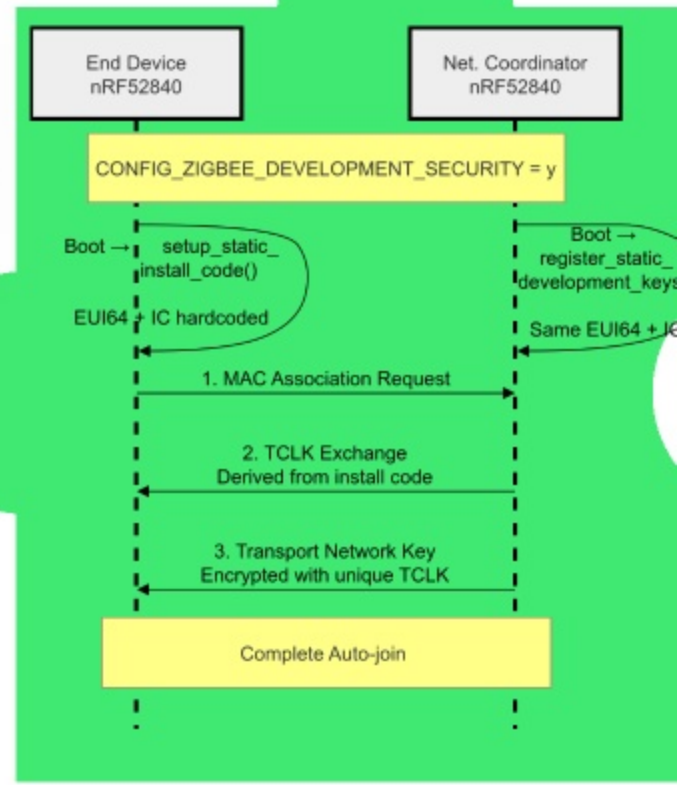


B

- Fundament Criptografic - TCLK & AES-128:
Autentificare bazată pe Install Code unic per dispozitiv (16 octeți aleatori + 2 octeți CRC-16/X-25 LE).
- Regimuri de lucru - Dev & Prod:
Identificarea diferențelor dintre parcurgerea automată și înrolarea restricționată în fereastră de 180 secunde.

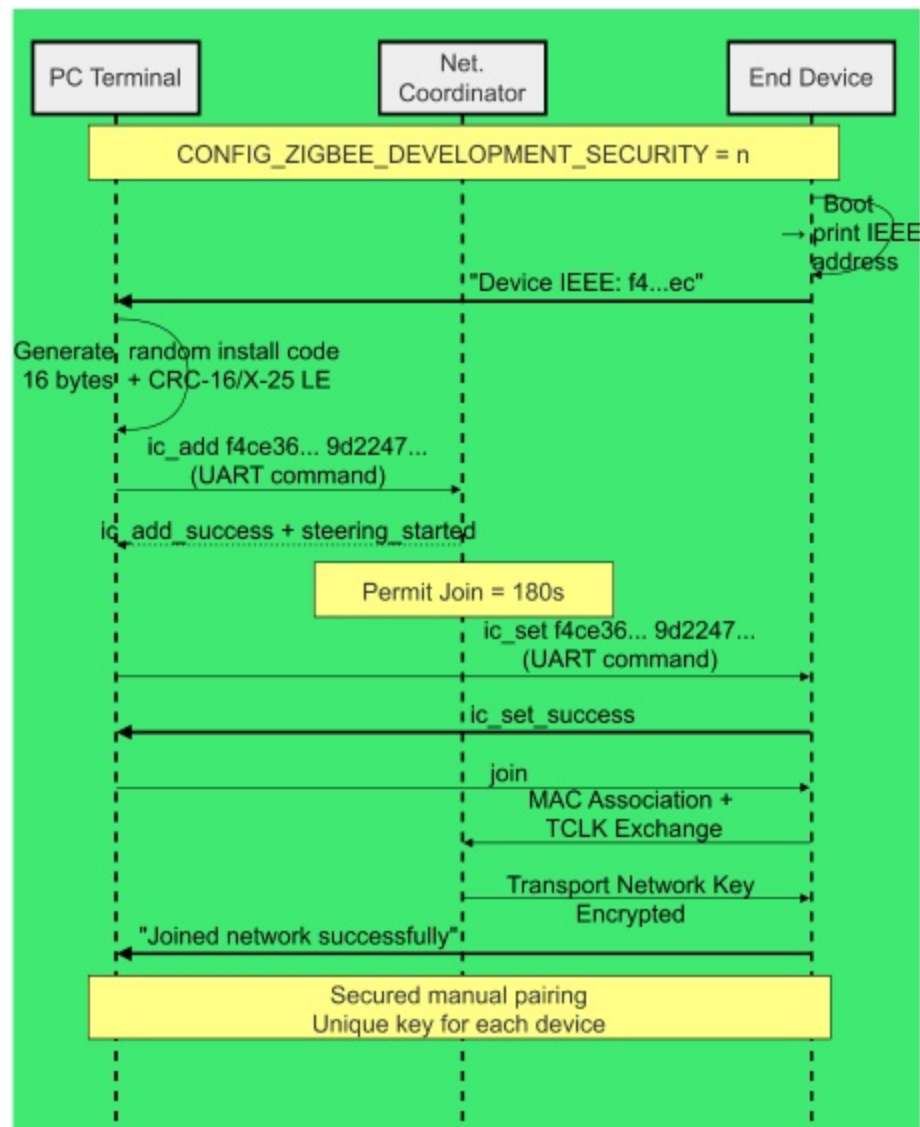
Autentificarea în Rețea

C



UART CLI
_set, join, factory_reset

Queues / Timers



dispozitiv (16 octeți aleator
LE).

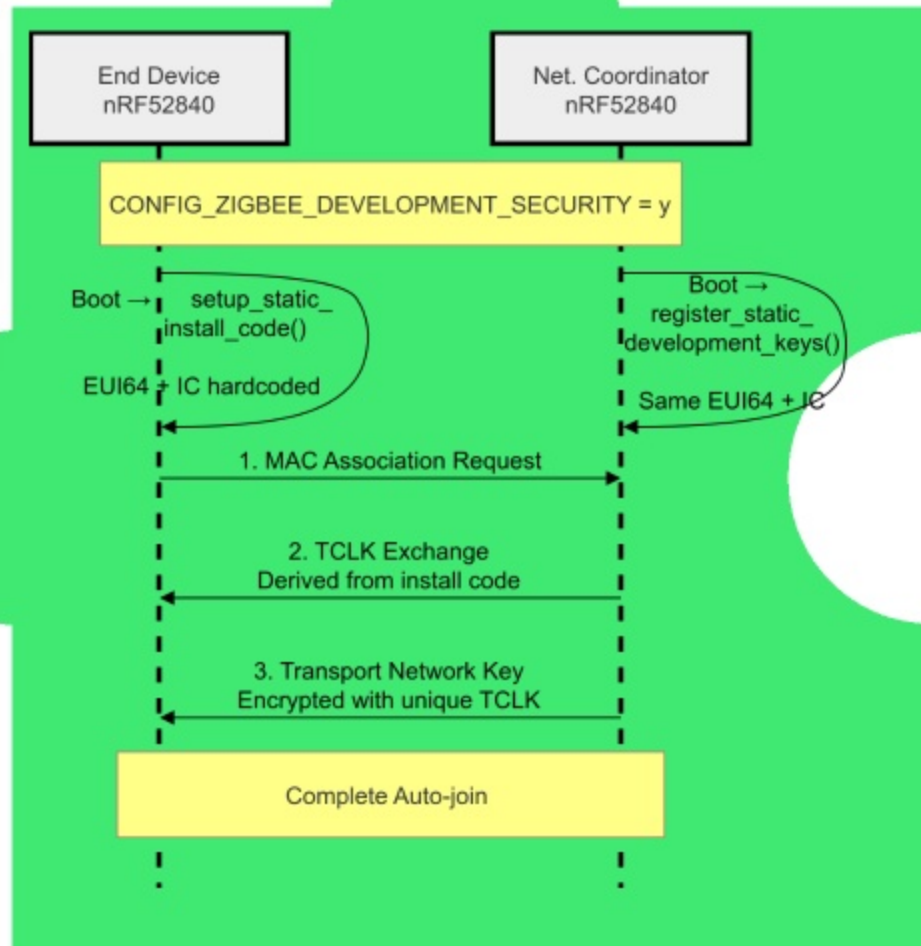
- Regimuri de lucru - Dev & Prod
Identificarea diferențelor din
automată și înrolarea restric
180 secunde.

Autentificarea

- **Fundament Criptografic - TCLK & AES-128:**
Autentificare bazată pe Install Code unic per dispozitiv (16 octeți aleatori + 2 octeți CRC-16/X-25 LE).
- **Regimuri de lucru - Dev & Prod:**
Identificarea diferențelor dintre parcurgerea automată și înrolarea restricționată în fereastră de 180 secunde.

Autentificarea în Rețea

C

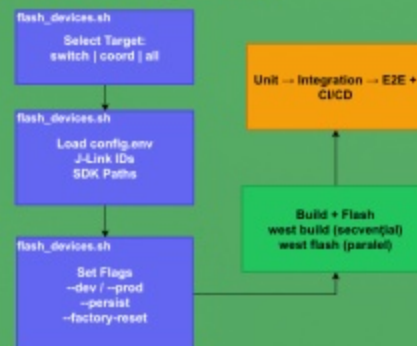


Decision Config Tree



Configurare Kconfig Flash CLI Automatizat

Fluxul de Programare



Mecanisme de Control și Configurare

- Control la Compilare prin Kconfig:
Comutare între 3 profile DEVELOPMENT_SECURITY, RESET_PERSISTENT, TEST_MODE), fără modificarea codului sursă.
- Mapare CLI și Protecție la Eroare:
Scriptul ./flash_devices.sh traduce argumentele CLI în opțiuni Kconfig și execută verificări de siguranță
ex.: avertisment automat la apelarea --prod fără --persist
- Execuție Optimizată
Compilare ordonată pentru stabilitatea mediului west, urmată de scriere simultană pe ambele plăci utilizând ID-urile unice J-Link.
- Depozit Unic de Configurare și Stare Curată:
Decuplare totală a parametrilor hardware prin config.env și suport pentru ștergerea integrală NVM prin --factory-reset.

```

george@george-KLUL-W039
=====
Mode: DEVELOPMENT (auto)
Storage: VOLATILE (reset)
Flash action: NORMAL
=====
Usage: ./flash_devices.sh [options]

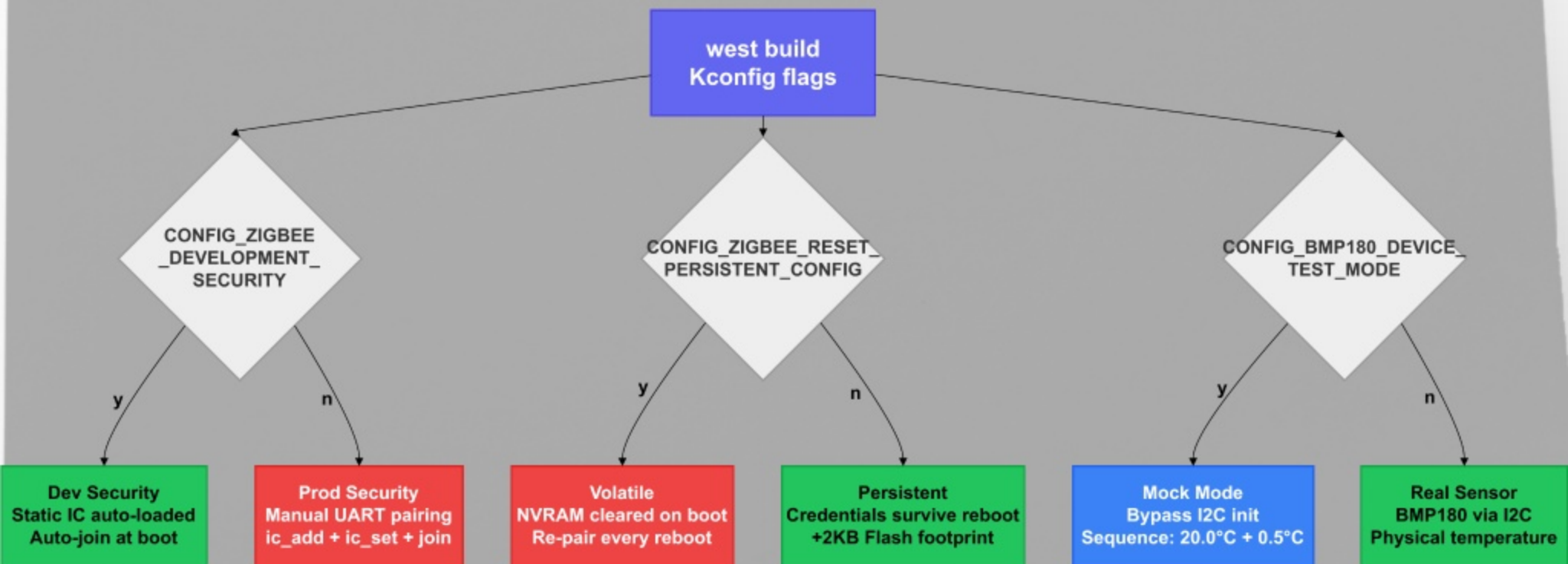
Targets:
  switch  Flash the switch
  coord   Flash the coordinator
  all      Build and flash all

Modes:
  --dev    Development mode
  --prod   Production mode

Options:
  --persist  Enable persistent storage
  --factory-reset  Factory reset the device

Examples:
  ./flash_devices.sh all
  ./flash_devices.sh switch
  ./flash_devices.sh coord
  ./flash_devices.sh all --persist
  
```

Decision Config Tree



Fluxul de Programare

flash_devices.sh

Select Target:
switch | coord | all

flash_devices.sh

Load config.env
J-Link IDs
SDK Paths

flash_devices.sh

Set Flags
--dev / --prod
--persist
--factory-reset

Unit → Integration → E2E +
CI/CD

Build + Flash
west build (secvențial)
west flash (paralel)



Mecanisme de Control și Configurare

- Control la Compilare prin Kconfig:
Comutare între 3 profile DEVELOPMENT_SECURITY, RESET_PERSISTENT, TEST_MODE), fără modificarea codului sursă.
- Mapare CLI și Protecție la Eroare:
Scriptul ./flash_devices.sh traduce argumentele CLI în opțiuni Kconfig și execută verificări de siguranță
ex.: avertisment automat la apelarea --prod fără --persist
- Execuție Optimizată
Compilare ordonată pentru stabilitatea mediului west, urmată de scriere simultană pe ambele plăci utilizând ID-urile unice J-Link.
- Depozit Unic de Configurare și Stare Curată:
Decuplare totală a parametrilor hardware prin config.env și suport pentru ștergerea integrală NVM prin --factory-reset.

```
george@george-KLVL-WXX9:~/Git-Projects/ZigBee-Gateway_nRF52840-DK$ ./flash_devices.sh -h

=====
Mode: DEVELOPMENT (auto-join with static key)
Storage: VOLATILE (resets NVRAM on boot)
Flash action: NORMAL
=====

Usage: ./flash_devices.sh {switch|coord|all} [--dev|--prod] [--persist] [--factory-reset]

Targets:
switch      Flash the End Device (BMP180 sensor)
coord       Flash the Network Coordinator
all         Build and flash both devices

Modes:
--dev       Development mode: auto-join with static key (default)
--prod      Production mode: manual UART pairing required

Options:
--persist   Enable NVRAM persistence (do not erase network config on boot)
--factory-reset Force flash erase (clean NVRAM settings before flashing)

Examples:
./flash_devices.sh all           # dev mode (default)
./flash_devices.sh all --prod    # production mode (volatile)
./flash_devices.sh all --prod --persist # production mode (persistent)
./flash_devices.sh all --prod --persist --factory-reset # clean settings, then persistent
```

Rolul fiecărui nivel

Unit test:

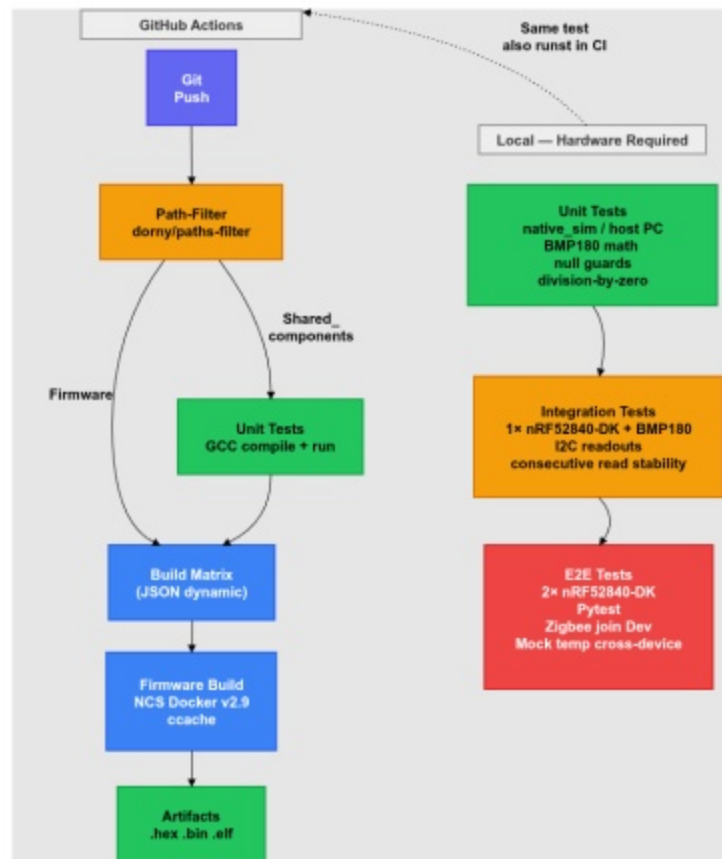
- verifică logica în izolare, fără hardware, rulează și în CI

Integration test:

- verificarea driverului pe placa nRF

E2E:

- tratarea nedeterminismului radio în mediul de CI/CD

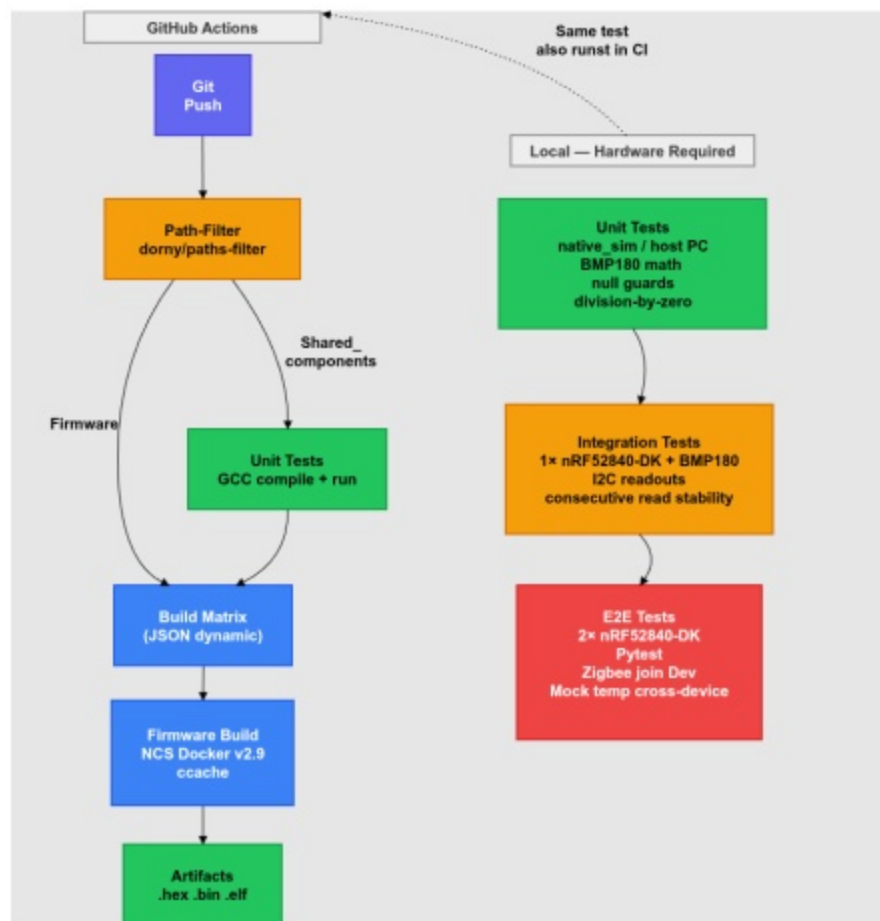


rui nivel

re, fără hardware, rulează

de placa nRF

mului radio în mediul de





Concluzii

Concluzii și Lecții

- Necesitatea cozilor de mesaje între I2C și contextul ZBOSS.
- Automatizarea testării și a programării salvează timp și garantează că hardware-ul se comportă identic la fiecare build.
- Proiectul poate fi extins ușor către noi senzori sau funcționalități

- Separarea profilelor de com prin Kconfig permite schimb rapidă fără a modifica linii d
- Arhitectura sistemului trebuie proiectată în jurul raportului consum dintre starea pasivă – **1.5 μ A în Deep Sleep** și transmisia RF **~6.4 mA**. Autonomia nodului depinde de timpul petrecut în stare a

Concluzii și Lecții

- Necesitatea cozilor de mesaje între I2C și contextul ZBOSS.
- Automatizarea testării și a programării salvează timp și garantează că hardware-ul se comportă identic la fiecare build.
- Proiectul poate fi extins ușor către noi senzori sau funcționalități

- Separarea prot...
- prin Kconfig pe...
- rapidă fără a m...

- Separarea profilelor de compilare prin Kconfig permite schimbare rapidă fără a modifica linii de cod.
- Arhitectura sistemului trebuie proiectată în jurul raportului de consum dintre starea pasivă **~0.4 – 1.5 μ A în Deep Sleep** și transmisia RF **~6.4 mA**.
Autonomia nodului depinde strict de timpul petrecut în stare activă.

